

Từ trò chơi trừ động theo bội k đến một lớp bài toán trò chơi tổ hợp

Cao Qinxiang

Trường Trung học số 2 trực thuộc Đại học Sư phạm Hoa Đông

2009

Nguồn gốc: Từ trò chơi trừ động theo bội k đến một lớp bài toán trò chơi tổ hợp

IOI China candidate team papers / 2009 / Cao Qinxiang

Abstract

Trò chơi tổ hợp là trò chơi hai người, luân phiên thao tác, thông tin hoàn toàn. Bài viết bắt đầu từ trò chơi trừ động theo bội k (k -based dynamic subtraction game), sau đó bàn về những phương pháp thường dùng để giải bài toán trò chơi tổ hợp trong tin học: định lý trạng thái N/P , quy hoạch động, đơn điệu, tìm kiếm có nhớ, hàm Sprague–Grundy, tổng Nim, tích Nim, đối xứng và tham lam. Tác giả cũng tối ưu thuật toán cho trò chơi trừ động từ $\mathcal{O}(S^3)$ xuống $\mathcal{O}(S)$, đưa ra thuật toán tính tích Nim trong $\mathcal{O}(\log^2 x)$, đồng thời phân tích lại lời giải chính thức của một bài BOI 2008 để chỉ ra sai lệch trong ước lượng độ phức tạp và cải thiện về $\mathcal{O}(n^2)$.

Từ khóa. Trạng thái N/P ; đơn điệu; hàm SG; tổng Nim; tích Nim; phân tích đối xứng; phân tích tham lam; phân tích độ phức tạp.

1 Mở đầu

1.1 Định nghĩa trò chơi tổ hợp

Từ “trò chơi” bao trùm rất rộng: kéo búa bao, bài, cờ tướng, cờ vây, Nim, v.v. Trong bài viết này, một trò chơi được xem là trò chơi tổ hợp nếu thỏa các điều kiện sau:

1. Có đúng hai người chơi.
2. Hai bên luân phiên thao tác; không có lượt thao tác đồng thời.
3. Ở mọi thời điểm, tập thao tác hợp lệ của người đang đi là một tập hữu hạn và xác định; không có yếu tố ngẫu nhiên.
4. Thông tin hoàn toàn: cả hai bên đều biết trạng thái ban đầu và toàn bộ lịch sử thao tác.
5. Một số trạng thái được quy định là trạng thái kết thúc thắng, người nhận được chúng thắng; một số trạng thái được quy định là trạng thái kết thúc thua, người nhận được chúng thua. Bài viết chỉ xét các trò chơi luôn phân thắng thua sau hữu hạn lượt, không xét hòa.

Vì vậy trò chơi một người như đẩy hộp, trò có thao tác đồng thời như kéo búa bao, trò có gieo xúc xắc, hoặc trò có thông tin ẩn đều nằm ngoài phạm vi này.

Một trò chơi tổ hợp thường được mô tả bằng ánh xạ

$$f : X \rightarrow \mathcal{P}(X),$$

trong đó X là tập trạng thái, $\mathcal{P}(X)$ là tập các tập con của X , và $f(x)$ là tập trạng thái có thể đạt được sau một nước đi từ x . Ta gọi $y \in f(x)$ là một hậu duệ trực tiếp của x . Nói cách khác, trò chơi có thể được hiểu như một đồ thị có hướng không chu trình $G = (V, E)$, với $V = X$ và

$$E = \{(x, y) : y \in f(x)\}.$$

1.2 Những câu hỏi cần trả lời

Với một trò chơi hai người, thông thường cần trả lời hai câu hỏi: ai có chiến lược thắng, và chiến lược thắng đó là gì. Trong bài toán tin học, yêu cầu tương ứng có ba mức:

1. Chỉ xác định ai có chiến lược thắng.
2. Tìm một nước đi thắng trong một lượt.
3. Duy trì nước đi thắng qua nhiều lượt, như trong bài tương tác.

1.3 Thước đo độ phức tạp

Với bài toán trò chơi, ngoài kích thước dữ liệu vào, còn có ba đại lượng tự nhiên: $S(x)$ là số hậu duệ trực tiếp hoặc gián tiếp của trạng thái hiện tại, $D(x)$ là lượng dữ liệu cần để mô tả trạng thái hiện tại, và $T(x)$ là số lượt tối đa còn lại trong trường hợp xấu nhất. Vì mô tả trạng thái phải phân biệt được nó với các trạng thái khác, thường có

$$D(x) = \Theta(\log S(x)).$$

Mặt khác, do đồ thị trò chơi không có chu trình, với mọi $y \in f(x)$ ta có

$$S(x) \geq S(y) + 1,$$

nên $S(x) = \Omega(T(x))$. Có trò chơi đạt $S(x) = \Theta(T(x))$, chẳng hạn trò lấy đá đơn giản. Nhưng nhiều trò chơi không như vậy: trong trò chơi trừ động theo bội k , $S(x) = \Theta(T(x)^2)$; trong Nim k đồng, $S(x) = \Theta(T(x)^k)$; trong Green Hackenbush, $S(x) = \Theta(2^{T(x)})$.

Do đó bài viết thường dùng $T(e)$, với e là trạng thái ban đầu, làm thước đo thời gian tự nhiên của trò chơi. Với bài tương tác, thuật toán trực tuyến lý tưởng có dạng $\mathcal{O}(T(e)) - \mathcal{O}(1)$: tiền xử lý tuyến tính theo thời lượng trò chơi, sau đó mỗi lượt trả lời hằng số.

1.4 Ý nghĩa trong thi Tin học

Bài toán trò chơi vừa kiểm tra nguyên lý cơ bản của quy hoạch động, vừa đòi hỏi khả năng vận dụng linh hoạt nhiều kiểu phân tích: đơn điệu, đối xứng, tham lam, công thức số học, hàm SG và mô hình hóa trạng thái. Vì không có một công thức cố định dùng được cho mọi bài, đây là dạng bài tổng hợp rất thích hợp để kiểm tra năng lực phân tích. Trong vài năm gần đây, đặc biệt là vào thời điểm bài viết gốc ra đời, các bài trò chơi xuất hiện ngày càng nhiều ở các kỳ thi quốc tế và trên các judge lớn, nên có thể xem trò chơi tổ hợp là một hướng nóng mới trong thi Tin học.

2 Bài toán xuất phát

Trò chơi trừ động theo bội k . Cho số nguyên $S \geq 2$. Người đi trước trừ khỏi S một số x , ít nhất là 1 và nhỏ hơn S . Sau đó hai bên luân phiên trừ khỏi S một số nguyên dương, nhưng số bị trừ ở lượt hiện tại không được vượt quá k lần số đối thủ vừa trừ ở lượt trước. Ai làm S trở thành 0 thì thắng. Hỏi ai có chiến lược thắng.

Ta biểu diễn trạng thái bằng cặp có thứ tự (m, n) : hiện còn m , và người đang đi được trừ nhiều nhất n . Trạng thái ban đầu là $(S, S - 1)$.

Với $k = 1$ và $k = 2$, bài viết đội tuyển quốc gia năm 2002 đã cho kết quả đúng. Khi $k = 1$, hậu thủ thắng khi và chỉ khi S là lũy thừa của 2. Khi $k = 2$, hậu thủ thắng khi và chỉ khi S là một số Fibonacci. Việc kiểm tra hai điều kiện này có thể làm trong $\mathcal{O}(\log S)$, hoặc $\mathcal{O}(\log^2 S)$ nếu tính cả số học nhiều chữ số. Phần sau trình bày cách xử lý k tổng quát và cách duy trì chiến lược thắng.

3 Nền tảng: trạng thái N/P và quy hoạch động

3.1 Khái niệm trạng thái N và P

Trạng thái N là trạng thái mà người sắp đi có chiến lược thắng (*Next player wins*). Trạng thái P là trạng thái mà người vừa đi có chiến lược thắng (*Previous player wins*).

Ví dụ, trong trò lấy đá có 21 viên, mỗi lượt lấy 1, 2 hoặc 3 viên, ai lấy viên cuối thắng. Trạng thái còn 0 viên là P . Các trạng thái còn 1, 2, 3 viên là N , vì người sắp đi có thể lấy hết. Trạng thái còn 4 viên là P , vì mọi nước đi đều đưa đối thủ tới một trạng thái N . Suy ra các trạng thái 0, 4, 8, 12, ... là P ; trạng thái ban đầu 21 viên là N , nên người đi trước thắng.

Định lý trạng thái N/P. Một trạng thái là P nếu nó là trạng thái kết thúc thắng hoặc mọi hậu duệ của nó đều là N . Một trạng thái là N nếu nó là trạng thái kết thúc thua hoặc có ít nhất một hậu duệ là P .

3.2 Quy hoạch động tổng quát

Từ định lý trên, có thể tính nhãn N/P bằng quy hoạch động trên thứ tự topo ngược của đồ thị trò chơi:

1. Đánh dấu trạng thái kết thúc thắng là P , trạng thái kết thúc thua là N .
2. Với trạng thái chưa biết mà mọi hậu duệ đã biết đều là N , đánh dấu nó là P .
3. Với trạng thái chưa biết có một hậu duệ là P , đánh dấu nó là N .
4. Nếu hai bước trước không tạo nhãn mới thì dừng; ngược lại lặp lại.

Với đồ thị $G(V, E)$, cách làm này có độ phức tạp $\mathcal{O}(|E|)$. Đây là phương pháp phổ quát nhưng thường không đủ nhanh. Với trò chơi trừ động theo bội k , ta có $|E| = \Theta(S^3)$ và $T(e) = \Theta(S)$, nên thuật toán $\mathcal{O}(S^3)$ là quá lớn. Phần tiếp theo tối ưu nó về tuyến tính theo S .

4 Tối ưu hóa dựa trên quy hoạch động

4.1 Đơn điệu trạng thái trong trò chơi trừ động

Đặt

$$\text{NP}(m, n) = \begin{cases} 1, & (m, n) \text{ là trạng thái } N, \\ 0, & (m, n) \text{ là trạng thái } P. \end{cases}$$

Trong trò chơi trừ động, $\text{NP}(m, n)$ đơn điệu không giảm theo n . Nếu $\text{NP}(m_0, n_0) = 1$, thì với mọi $n > n_0$, trạng thái (m_0, n) cũng là N , vì người chơi có ít nhất toàn bộ các nước đi hợp lệ ở (m_0, n_0) và có thể đi tới cùng trạng thái thắng.

Do đó chỉ cần lưu

$$f(m) = \min\{n : \text{NP}(m, n) = 1\}.$$

Vì $NP(m, m) = 1$, giá trị này luôn tồn tại. Theo định nghĩa, hàng m của bảng trạng thái có dạng: các cột $n < f(m)$ là P , các cột $n \geq f(m)$ là N .

Khi từ (m, n) trừ r , trạng thái mới là $(m - r, kr)$. Do đó (m, n) là N khi tồn tại $1 \leq r \leq n$ sao cho $(m - r, kr)$ là P . Suy ra công thức chuyển:

$$f(m) = \min\{n : f(m - n) > kn\},$$

với quy ước $f(0) = +\infty$, vì còn 0 là trạng thái P bất kể giới hạn trừ ở lượt sau. Kiểm tra trực tiếp từng n cho mỗi m tạo thuật toán $\mathcal{O}(S^2)$. Hậu thủ thắng ở trạng thái ban đầu khi và chỉ khi $f(S) = S$.

4.2 Đơn điệu quyết định và ngăn xếp

Thuật toán $\mathcal{O}(S^2)$ vẫn chưa khai thác hết cấu trúc. Nếu vẽ bảng $NP(m, n)$ và tô đen các ô P , mỗi hàng tạo một đoạn liên tiếp giống như một bức tường. Công thức

$$f(m) = \min\{n : f(m - n) > kn\}$$

có thể hiểu hình học như sau: từ ô $(m, 0)$, kẻ một đường thẳng có hệ số góc $k - 1$; $f(m)$ là khoảng cách tới bức tường đầu tiên mà đường này gặp.

Các đường thẳng này song song nhau và khi m tăng thì chúng dịch dần xuống phải. Mỗi bức tường cố định và có đầu phải hữu hạn. Vì thế nếu một bức tường không chặn được đường hiện tại, nó cũng sẽ không chặn được bất kỳ đường nào sau này. Ta dùng ngăn xếp để lưu các bức tường còn hữu ích:

1. Khi tính $f(m)$, kiểm tra lần lượt bức tường trên đỉnh ngăn xếp.
2. Nếu nó không chặn được đường xuất phát từ $(m, 0)$, loại nó khỏi ngăn xếp.
3. Nếu nó chặn được, dừng và suy ra $f(m)$.
4. Từ $f(m)$, tạo bức tường mới nếu có, rồi đưa vào ngăn xếp.

Mỗi bức tường được đưa vào ngăn xếp đúng một lần và bị loại nhiều nhất một lần. Vì mỗi m tạo nhiều nhất một bức tường, tổng thời gian là $\mathcal{O}(S)$. Đây là thuật toán tuyến tính theo $T(e)$.

4.3 Tìm và duy trì chiến lược thắng

Sau khi biết toàn bộ $f(m)$, nếu trạng thái hiện tại (m, n) là N , tức $n \geq f(m)$, thì trừ đúng $f(m)$ là một nước thắng, đồng thời là nước thắng nhỏ nhất. Vì vậy trò chơi trực tuyến có thể xử lý trong $\mathcal{O}(S) - \mathcal{O}(1)$.

Với $k = 1$ hoặc $k = 2$ có thể có thuật toán nhanh hơn nhờ công thức số học riêng, nhưng phần cốt lõi của trường hợp tổng quát là đơn điệu trạng thái và đơn điệu quyết định trong quy hoạch động.

4.4 Tìm kiếm có nhớ

Quy hoạch động có thể triển khai bằng truy hồi có nhớ thay vì duyệt topo. Từ định lý N/P ,

$$NP(x) = \begin{cases} P, & \forall y \in f(x), NP(y) = N, \\ N, & \exists y \in f(x), NP(y) = P. \end{cases}$$

Khi đang tìm kiếm, chỉ cần gặp một hậu duệ P là biết trạng thái hiện tại là N , không cần xét các hậu duệ còn lại. Hai chiến lược thường dùng là:

1. Dùng heuristic để thử trước nước đi có nhiều khả năng là nước thắng, hoặc thử trước trạng thái con dễ phán đoán.

2. Dùng ngẫu nhiên để xáo thứ tự nước đi, tránh dữ liệu đặc biệt làm tìm kiếm rơi vào nhánh xấu.

Tìm kiếm có nhớ có thể tránh nhiều trạng thái không xuất hiện trong thực tế và được lợi nhờ cắt nhánh. Ngược lại, nhiều tối ưu dựa trên đơn điệu quyết định chỉ thuận tiện với cách tính lặp.

4.5 Quan sát quy luật: trò knight của CEOI 2008

Trò knight của CEOI 2008 đặt K quân mã trên bàn cờ $N \times N$. Mỗi lượt, người chơi phải di chuyển tất cả các quân mã còn di chuyển được theo một trong bốn hướng hợp lệ đã cho; quân nào không đi được thì đứng yên. Ai tới lượt mà không quân nào đi được thì thua.

Trước hết xét trường hợp chỉ có một quân mã ở (x, y) . Vì sau mỗi nước $x + y$ đều giảm, đồ thị trạng thái là DAG và có thể dùng quy hoạch động $\mathcal{O}(N^2)$. Nhưng N lớn nên lời giải như vậy chỉ dùng để tạo bảng nhỏ và quan sát quy luật. Các bảng N/P thay đổi theo $N \bmod 4$; sau khi tách theo phần dư, mẫu thắng thua trở nên đều đặn và có thể xác định trong $\mathcal{O}(1)$. Công thức một quân chưa giải quyết trực tiếp trường hợp nhiều quân, nhưng nó cho biết đại lượng nào cần đo khi các quân cùng di chuyển theo từng vòng.

Với K quân mã, chỉ biết từng quân thắng hay thua chưa đủ. Người chơi muốn thắng ở quân thắng càng muộn càng tốt, và thua ở quân thua càng sớm càng tốt. Gọi $\text{Round}(x, y)$ là số lượt tới khi quân ở (x, y) phân thắng thua, và đặt

$$RR(x, y) = \begin{cases} \text{Round}(x, y), & (x, y) \text{ là } N, \\ -\text{Round}(x, y), & (x, y) \text{ là } P. \end{cases}$$

Nếu (x^*, y^*) chạy qua các hậu duệ của (x, y) , công thức chuyển có dạng

$$RR(x, y) = \begin{cases} -\min RR(x^*, y^*) + 1, & (x, y) \text{ là } N, \\ -\min RR(x^*, y^*) - 1, & (x, y) \text{ là } P. \end{cases}$$

Quân có $|RR|$ lớn nhất quyết định kết quả chung: giá trị dương nghĩa là tiên thủ thắng, giá trị âm nghĩa là hậu thủ thắng.

Trực giác chiến lược là: với một quân mà mình có thể thắng, người chơi muốn trì hoãn chiến thắng đó càng lâu càng tốt; với một quân chắc thua, người chơi muốn kết thúc nó càng nhanh càng tốt. Nếu hai bên đều chỉ có thể thắng ở một số quân riêng lẻ, quân có thời điểm phân định muộn nhất sẽ là quân quyết định toàn cục. Dấu của RR ghi ai thắng trên quân đó, còn trị tuyệt đối ghi độ muộn của kết cục.

Bảng thực nghiệm cho thấy

$$\lim_{x, y \rightarrow \infty} \frac{\text{Round}(x, y)}{x + y} = \frac{1}{2},$$

và sau khi trừ $\lfloor x/2 \rfloor + \lfloor y/2 \rfloor$, phần lõi lặp theo khối 4×4 :

$$\begin{pmatrix} 2 & -1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 1 & 0 & 1 & 2 \end{pmatrix}.$$

Lớp ngoài cùng vẫn phụ thuộc $N \bmod 4$. Có thể viết công thức $\mathcal{O}(1)$, nhưng cách thực dụng hơn là dùng công thức cho phần trong và để quy hoạch động xử lý lớp ngoài, đạt $\mathcal{O}(K + N)$, đủ tốt khi K cùng bậc với N .

5 Hàm SG, tổng Nim và tích Nim

5.1 Hàm SG và tổng Nim

Với tập hữu hạn S gồm các số nguyên không âm, đặt

$$\text{mex}(S) = \min\{x : x \notin S, x \geq 0\}.$$

Với trò chơi không có trạng thái kết thúc thua, hàm Sprague–Grundy được định nghĩa bởi

$$\text{SG}(x) = \begin{cases} 0, & x \text{ là trạng thái kết thúc thắng,} \\ \text{mex}\{\text{SG}(y) : y \in f(x)\}, & \text{trường hợp khác.} \end{cases}$$

Từ định lý N/P , $\text{SG}(x) = 0$ khi và chỉ khi x là trạng thái P .

Tổng Nim là phép xor trên các số nguyên không âm, ký hiệu $\oplus$. Nó có các tính chất giao hoán, kết hợp, $a \oplus a = 0$, $a \oplus 0 = a$, nên phương trình $a \oplus x = b$ có nghiệm duy nhất $x = a \oplus b$.

Định lý SG. Nếu trò chơi tổng

$$G = G_1 + G_2 + \dots + G_n$$

cho phép mỗi lượt chọn đúng một trò con và đi một nước trong đó, thì với $x = (x_1, x_2, \dots, x_n)$,

$$\text{SG}(x) = \text{SG}(x_1) \oplus \text{SG}(x_2) \oplus \dots \oplus \text{SG}(x_n).$$

Định lý này biến việc tính trên tích Descartes khổng lồ thành việc tính từng trò con. Với Nim cổ điển có k đồng $x_1, \dots, x_k$, trò một đồng có $\text{SG}(x) = x$. Do đó tiên thủ thắng khi và chỉ khi

$$x_1 \oplus x_2 \oplus \dots \oplus x_k \neq 0.$$

5.2 Chiến lược thắng trong Nim và duy trì trực tuyến

Giả sử $X = x_1 \oplus \dots \oplus x_k \neq 0$. Chọn một đồng i mà bit cao nhất của X cũng bật trong x_i , rồi đặt

$$y_i = x_i \oplus X, \quad y_j = x_j \quad (j \neq i).$$

Khi đó $y_i < x_i$ và xor của các đồng mới bằng 0, nên đây là nước thắng.

Nếu mỗi lượt đều tính lại từ đầu, chi phí là $\mathcal{O}(k)$. Trong bài tương tác, có thể duy trì tổng xor trong $\mathcal{O}(1)$ sau mỗi lượt, vì nhiều nhất hai đồng thay đổi giữa hai lượt của ta. Để tìm nhanh đồng chứa bit cần thiết, dùng $\log(\max x_i)$ danh sách liên kết hai chiều, mỗi danh sách lưu những đồng có bit tương ứng bằng 1. Khi cập nhật một đồng, sửa các danh sách theo các bit của nó, đạt $\mathcal{O}(\log \max x_i)$ mỗi lượt sau tiền xử lý $\mathcal{O}(k \log \max x_i)$.

5.3 Lồng ghép định lý SG

Định lý SG có thể được lồng lồng nhau. Xét trò có n quân cờ xếp thành hàng; mỗi lượt lấy một quân hoặc hai quân kề nhau, ai lấy quân cuối thắng. Nếu lấy ở giữa, hàng cờ tách thành hai trò độc lập. Vì vậy

$$\text{SG}(n) = \text{mex}\{\text{SG}(n-1), \text{SG}(n-2), \text{SG}(i) \oplus \text{SG}(n-i-1), \text{SG}(i) \oplus \text{SG}(n-i-2)\}.$$

Với $n > 0$, thực ra $\text{SG}(n) > 0$: người đi trước chỉ cần lấy ở giữa để tạo hai nửa đối xứng. Ví dụ này nhắc rằng không nên bó buộc suy nghĩ vào việc tính toán bộ bảng N/P hoặc SG nếu có một lập luận cấu trúc đơn giản hơn.

5.4 Tổng Nim bậc N

Trong Nim bậc N , mỗi lượt được chọn từ 1 đến N đồng không rỗng và lấy tùy ý một số dương ở mỗi đồng được chọn. Hậu thủ thắng khi và chỉ khi ở mỗi bit nhị phân, số đồng có bit đó bằng 1 là bội của $N + 1$. Kết luận này tương tự định lý SG, nhưng khi $N > 1$ nó chỉ cho tiêu chuẩn thắng thua và nước thắng; nó không cho một giá trị SG duy nhất để có thể lồng tiếp.

5.5 Tích Nim từ trò lật đồng xu

Trò lật đồng xu I đặt đồng xu trên bảng tọa độ $(0, 0)$ tới (n, m) . Mỗi lượt lật hai đồng xu cùng hàng hoặc cùng cột, trong đó đồng xu có tọa độ lớn hơn phải đang ngửa và bị lật úp. Khi chỉ có một đồng xu ngửa tại (x, y) , trò tương đương tổng của hai đồng Nim, nên

$$SG(x, y) = x \oplus y.$$

Trò lật đồng xu II thay thao tác bằng việc lật bốn đỉnh của một hình chữ nhật:

$$(x, y), (x, b), (a, y), (a, b), \quad 0 \leq a < x, 0 \leq b < y,$$

trong đó (x, y) phải đang ngửa và bị lật úp. Khi chỉ có một đồng xu ngửa, hàm SG phức tạp hơn. Ta định nghĩa tích Nim bằng

$$x \otimes y = \text{mex}\{(a \otimes y) \oplus (x \otimes b) \oplus (a \otimes b) : 0 \leq a < x, 0 \leq b < y\}.$$

Đây là giá trị SG của một đồng xu ở (x, y) trong trò II.

Định lý Tartan. Với tích trò chơi $G = G_1 \cdot G_2 \cdots G_n$ được định nghĩa sao cho một lượt chọn đồng thời hậu duệ trong một số tọa độ và tổng hóa tất cả trạng thái sinh ra theo quy tắc tích, ta có

$$SG(x_1, \dots, x_n) = SG(x_1) \otimes SG(x_2) \otimes \cdots \otimes SG(x_n).$$

Phép $\otimes$ có đơn vị 1, giao hoán, kết hợp, và phân phối trên $\oplus$:

$$(x \oplus y) \otimes z = (x \otimes z) \oplus (y \otimes z).$$

5.6 Tính tích Nim trong thời gian bình phương log

Tính trực tiếp từ định nghĩa mex rất chậm. Thuật toán nhanh dùng các tính chất sau. Với $x, y < 2^{2^a}$:

$$x \otimes 2^{2^a} = 2^{2^a} x, \quad x \otimes y < 2^{2^a}, \quad 2^{2^a} \otimes 2^{2^a} = \frac{3}{2} 2^{2^a}.$$

Ví dụ:

$$\begin{aligned} 24 \otimes 17 &= (16 \oplus 8) \otimes (16 \oplus 1) \\ &= (16 \otimes 16) \oplus (16 \otimes 8) \oplus (16 \otimes 1) \oplus (8 \otimes 1) \\ &= 24 \oplus 128 \oplus 16 \oplus 8 = 128. \end{aligned}$$

Một ví dụ khác:

$$\begin{aligned} 8 \otimes 8 &= (2 \otimes 4) \otimes (2 \otimes 4) \\ &= (2 \otimes 2) \otimes (4 \otimes 4) = 3 \otimes 6 \\ &= (1 \oplus 2) \otimes (4 \oplus 2) = 4 \oplus 2 \oplus 8 \oplus 3 = 13. \end{aligned}$$

Thuật toán đệ quy gồm hai hàm. Hàm $\text{NimMulti}(x, y)$ tính $x \otimes y$, còn $\text{NimMultiPower}(x, y)$ xử lý riêng trường hợp x là lũy thừa của 2.

$\text{NimMulti}(x, y)$.

1. Nếu $x < y$, đổi chỗ hai đối số.
2. Nếu $x < 16$, tra bảng.
3. Tìm a sao cho $2^{2^a} \leq x < 2^{2^{a+1}}$, đặt $M = 2^{2^a}$.
4. Viết $x = pM + q$, $y = sM + t$, với $0 \leq q, t < M$.
5. Tính

$$c_1 = \text{NimMulti}(p, s), \quad c_2 = \text{NimMulti}(p, t) \oplus \text{NimMulti}(q, s), \quad c_3 = \text{NimMulti}(q, t).$$

6. Trả về

$$(c_1 \oplus c_2)M \oplus c_3 \oplus \text{NimMultiPower}(M/2, c_1).$$

$\text{NimMultiPower}(x, y)$.

1. Nếu $x < 16$, tra bảng.
2. Tìm a sao cho $2^{2^a} \leq x < 2^{2^{a+1}}$, đặt $M = 2^{2^a}$.
3. Viết $x = PM$, $y = SM + T$.
4. Tính

$$d_1 = \text{NimMultiPower}(P, S), \quad d_2 = \text{NimMultiPower}(P, T).$$

5. Trả về

$$M(d_1 \oplus d_2) \oplus \text{NimMultiPower}(M/2, d_1).$$

Gọi $f(a)$ là thời gian xấu nhất của NimMulti khi $0 \leq x, y < 2^{2^{a+1}}$, và $g(a)$ là thời gian của hàm phụ trong cùng miền. Ta có

$$f(a) = 4f(a-1) + g(a-1), \quad g(a) = 3g(a-1).$$

Suy ra $g(a) = \mathcal{O}(3^a)$ và $f(a) = \mathcal{O}(4^a)$. Vì $a = \Theta(\log \log \max(x, y))$, thời gian là

$$\mathcal{O}(4^{\log \log x}) = \mathcal{O}(\log^2 x).$$

Tính đúng của hai công thức trả về đến từ phân phối của $\otimes$ trên $\oplus$ và các tính chất của lũy thừa 2^{2^a} . Cụ thể,

$$\begin{aligned} \text{NimMultiPower}(x, y) &= PM \otimes (SM + T) \\ &= (P \otimes M) \otimes ((S \otimes M) \oplus T) \\ &= ((M \otimes M) \otimes (P \otimes S)) \oplus (M \otimes (P \otimes T)) \\ &= M(d_1 \oplus d_2) \oplus \text{NimMultiPower}(M/2, d_1), \end{aligned}$$

và phép biến đổi tương tự cho NimMulti .

6 Phân tích không dựa hoàn toàn vào quy hoạch động

6.1 Đối xứng hóa

Đối xứng hóa là tìm cách đưa tình thế về dạng hai bên đối xứng; khi đối thủ có nước đi, ta cũng có nước đáp lại. Trong bài US Open 2008 The Ultimate Battle, chiến trường là lưới $W \times H$, hai người đứng ở hai ô khác hàng và khác cột. Mỗi lượt đi ngang hoặc dọc tùy ý nhưng không được vượt qua hàng hoặc cột của đối thủ, vì khi cùng hàng hoặc cùng cột thì đối thủ có thể bắn.

Điều kiện thắng có thể phân tích như Nim hai đống, nhưng cách nhìn đơn giản hơn là duy trì sau lượt của mình khoảng cách theo hàng và theo cột bằng nhau. Nếu đối thủ lùi theo một trục, ta tiến cùng số ô trên trục đó; nếu đối thủ tiến theo một trục, ta tiến cùng số ô trên trục kia. Vì ta luôn tiến, cuối cùng đối thủ hết nước hợp lệ.

6.2 Tham lam và bài BOI 2008 Game

Trong trò BOI 2008 Game, bàn cờ $n \times n$ có ô trắng đi được và ô đen là chướng ngại. Hai ô A, B là vị trí xuất phát của tiên thủ và hậu thủ. Mỗi lượt người chơi đi một ô lên, xuống, trái hoặc phải vào ô trắng. Nếu đi đúng vào ô đối thủ đang đứng, người đó được đi thêm một bước, tức “nhảy qua” đối thủ. Ai đi tới ô xuất phát của đối phương trước thì thắng, kể cả khi tới đó nhờ bước nhảy qua.

Nếu mô tả trạng thái trực tiếp bằng (x_1, y_1, x_2, y_2) và lượt đi, có $\mathcal{O}(n^4)$ trạng thái, quá lớn. Dấu hiệu tham lam là chữ “trước” trong điều kiện thắng: cả hai bên đều muốn tới điểm xuất phát của đối thủ càng nhanh càng tốt. Vì quãng đường ngắn nhất giữa A và B là như nhau cho cả hai, nếu không có luật nhảy qua thì tiên thủ thắng. Vậy hậu thủ chỉ có thể thắng nếu ép được việc nhảy qua trên một đường ngắn nhất; tiên thủ thắng nếu tránh được điều đó.

Lời giải chính thức của BOI dùng các lớp đường ngắn nhất. Gọi d là khoảng cách ngắn nhất giữa A và B , và $L_A[i]$ là tập ô nằm trên một đường ngắn nhất, cách A đúng i . Đặt $\text{NP}_A[i, j, k]$ là trạng thái tới lượt A , A đứng tại ô thứ j của $L_A[i]$, B đứng tại ô thứ k của $L_A[d - i]$. Tương tự $\text{NP}_B[i, j, k]$ cho lượt B . Lời giải chính thức ước lượng có $\mathcal{O}(n^3)$ trạng thái và mỗi chuyển trạng thái hằng số, nên tổng thời gian $\mathcal{O}(n^3)$, dùng mảng cuộn để còn $\mathcal{O}(n^2)$ bộ nhớ.

Vấn đề là ba chỉ số hình thức không bảo đảm mỗi chiều chỉ $\mathcal{O}(n)$. Trên một bàn cờ có hành lang uốn lượn, d có thể là $\Theta(n^2)$. Hơn nữa, $|L_A[i]|$ có thể lớn hơn tuyến tính theo những cách làm hỏng ước lượng. Tác giả mô tả một phản ví dụ kiểu “ống nước và bể nước”: bố trí $\mathcal{O}(\sqrt{n})$ hàng và $\mathcal{O}(\sqrt{n})$ cột các khối, mỗi khối kích thước $\mathcal{O}(\sqrt{n}) \times \mathcal{O}(\sqrt{n})$, rồi thiết kế các hành lang vào ra sao cho rất nhiều ô trong các khối cùng thuộc một lớp khoảng cách. Khi đó có lớp $|L_A[i]| = \Theta(n^{1.5})$, và các lớp đối xứng từ B cũng đạt cùng bậc. Vì vậy bộ nhớ có thể cần

$$\max_i |L_A[i]| \cdot |L_A[d - i]| = \Theta(n^3),$$

còn tổng số trạng thái trên các lớp có thể đạt cận dưới $\Omega(n^{3.5})$. Do đó ước lượng $\mathcal{O}(n^3)$ thời gian và $\mathcal{O}(n^2)$ bộ nhớ của lời giải chính thức không đúng về mặt lý thuyết.

Nói cách khác, để thật sự có thời gian $\mathcal{O}(n^3)$, cần chứng minh

$$\sum_i |L_A[i]| \cdot |L_A[d - i]| = \mathcal{O}(n^3),$$

và để thật sự có bộ nhớ $\mathcal{O}(n^2)$, cần chứng minh

$$\max_i |L_A[i]| \cdot |L_A[d - i]| = \mathcal{O}(n^2).$$

Phản ví dụ “ống nước và bể nước” phủ định cả hai ước lượng: các ống bảo đảm độ dài vào-ra cân bằng, còn các bể tạo ra nhiều ô cùng nằm trên một lớp khoảng cách, khiến số cặp trạng thái trên những lớp đối xứng tăng vượt xa dự đoán.

Tuy vậy, ý tưởng tham lam vẫn đúng. Ta tô đen mọi ô trắng không nằm trên đường ngắn nhất nào giữa A và B . Với mỗi i , tập $L_A[i]$ cùng ô đen và biên bản cờ tạo thành một đường khép kín chia vùng trắng thành phía gần A và phía xa A . Các ô liên tiếp của $L_A[i]$ chỉ kề với những đoạn liên tiếp trong $L_A[i + 1]$. Vì thế, với mỗi cặp i, j , tập các k làm cho $NP_A[i, j, k] = N$ là một đoạn liên tiếp trên vòng $L_A[d - i]$; với NP_B cũng vậy.

Thay vì lưu toàn bộ k , chỉ cần lưu hai biên $left[i, j]$ và $right[i, j]$ của đoạn N trên vòng. Nếu $left \leq right$, đoạn thẳng là $[left, right]$; nếu $left > right$, đoạn thẳng vòng qua đầu mảng. Các biên này được truy hồi đơn giản từ lớp trước. Vì mỗi ô xuất hiện trong đúng một lớp $L_A[i]$, tổng số cặp (i, j) là $\mathcal{O}(n^2)$, nên thời gian và bộ nhớ đều có thể đưa về $\mathcal{O}(n^2)$.

Kết luận thực tế của ví dụ này không phải là bác bỏ hướng tham lam. Sai sót nằm ở bước ước lượng số trạng thái của lời giải chính thức: với ràng buộc nhỏ và hằng số thấp, chương trình kiểu đó vẫn có thể chạy qua nhiều bộ kiểm thử, nhưng mảng cấp phát theo $\mathcal{O}(n^2)$ có thể tràn trên cấu hình xấu. Điểm cần giữ lại là tiêu chí “đến trước” giúp thoát khỏi DP bốn chiều, còn phần chứng minh độ phức tạp phải dựa trên cấu trúc lớp đường ngắn nhất sau khi loại bỏ các ô không thuộc đường ngắn nhất.

7 Tổng kết

Định lý trạng thái N/P và quy hoạch động dựa trên nó là khung phổ quát cho bài toán trò chơi. Nhưng hiệu quả thực sự phụ thuộc vào việc phát hiện cấu trúc riêng của bài: đơn điệu trạng thái, đơn điệu quyết định, đối xứng, tham lam, hoặc công thức rút ra từ quan sát.

Hàm SG là phiên bản mạnh hơn của phân loại N/P . Khi kết hợp với tổng Nim và tích Nim, nó có thể xử lý nhiều trò chơi phức tạp. Tuy nhiên, khó khăn của bài toán thường không nằm ở việc áp dụng định lý, mà ở bước nhận ra trò chơi đã cho có thể biểu diễn thành tổng hoặc tích của các trò con như thế nào.

A Trường hợp k bằng 1 của trò chơi trừ động

Dùng ký hiệu của phần chính. Đặt $\text{lowbit}_2(m)$ là bit thấp nhất khác 0 của m trong nhị phân:

$$\text{lowbit}_2(m) = 2^t, \quad 2^t \mid m, \quad 2^{t+1} \nmid m.$$

Khi $k = 1$, ta có

$$f(m) = \text{lowbit}_2(m).$$

Chứng minh bằng quy nạp. Với $0 < n < \text{lowbit}_2(m)$, phép trừ nhị phân cho

$$\text{lowbit}_2(m - n) = \text{lowbit}_2(n) \leq n.$$

Theo giả thiết quy nạp,

$$f(m - n) = \text{lowbit}_2(m - n) \leq n = kn,$$

nên $f(m) \geq \text{lowbit}_2(m)$. Mặt khác,

$$f(m - \text{lowbit}_2(m)) > \text{lowbit}_2(m),$$

vì nếu $m = \text{lowbit}_2(m)$ thì về trái là $f(0) = +\infty$; còn nếu không, xóa bit 1 thấp nhất làm bit thấp nhất mới nằm ở vị trí cao hơn. Từ công thức chuyển suy ra $f(m) = \text{lowbit}_2(m)$.

Vì trạng thái ban đầu thua khi và chỉ khi $S = f(S)$, hậu thủ thắng đúng khi S là lũy thừa của 2. Ở mọi trạng thái N , trừ $\text{lowbit}_2(m)$ là nước thắng.

B Trường hợp k bằng 2 của trò chơi trừ động

Gọi $F_0 = F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$. Theo định lý Zeckendorf, mọi số nguyên dương m có biểu diễn duy nhất thành tổng các số Fibonacci đôi một khác nhau và không kề nhau:

$$m = \sum_{j=1}^r F_{i_j}, \quad i_j + 2 \leq i_{j+1}.$$

Đặt $\text{lowbit}_F(m)$ là hạng nhỏ nhất trong biểu diễn này. Khi $k = 2$,

$$f(m) = \text{lowbit}_F(m).$$

Ý tưởng chứng minh song song với trường hợp nhị phân. Nếu $0 < n < \text{lowbit}_F(m)$, thì phần nhỏ nhất trong biểu diễn của $m - n$ xuất phát từ $\text{lowbit}_F(m) - n$, và hạng Fibonacci nhỏ nhất của nó không vượt quá hai lần n :

$$f(m - n) \leq 2n.$$

Ngược lại, nếu $\text{lowbit}_F(m) = F_i$, thì sau khi trừ F_i , hạng nhỏ nhất còn lại ít nhất là $F_{i+2} > 2F_i$, hoặc rơi vào $f(0) = +\infty$. Vì vậy công thức chuyển với $k = 2$ cho $f(m) = \text{lowbit}_F(m)$.

Suy ra hậu thủ thắng khi và chỉ khi S là số Fibonacci. Ở trạng thái N , trừ $\text{lowbit}_F(m)$ là nước thắng. Nếu không tính số học nhiều chữ số, việc tìm $\text{lowbit}_F(m)$ mất $\mathcal{O}(\log m)$.

C Một vài biến thể Nim

1. Có n bậc thang, mỗi bậc có một số quân. Mỗi lượt chuyển vài quân từ bậc i xuống bậc $i - 1$. Bậc 0 là mặt đất; ai đưa mọi quân xuống đất thì thắng.
2. Có các ô $0, 1, \dots, n - 1$ trên một hàng, một số quân nằm trên các ô. Mỗi lượt chuyển một quân sang ô có chỉ số nhỏ hơn; nhiều quân được phép cùng ô. Ai không đi được thì thua.
3. Trên mỗi hàng của một bàn cờ chữ nhật có đúng một quân đen và một quân trắng. Người A chỉ đi quân trắng, người B chỉ đi quân đen, mỗi lượt đi trong cùng hàng và không được vượt qua quân khác. Ai không đi được thì thua.
4. Có k đồng $x_1 \leq x_2 \leq \dots \leq x_k$. Mỗi lượt chọn một đồng và lấy bớt đá nhưng phải giữ thứ tự không giảm, không được hoán đổi đồng. Ai lấy viên cuối thắng.
5. Trên một cây có gốc, mỗi lượt xóa một cạnh; phần không còn nối với gốc bị xóa theo. Ai xóa cạnh cuối cùng thắng.

D Phác thảo chứng minh định lý SG

Với hai trò con, cần chứng minh

$$\text{SG}(x_1, x_2) = \text{SG}(x_1) \oplus \text{SG}(x_2).$$

Tính chất then chốt của xor là

$$p \oplus q = \text{mex}\{a \oplus q : a < p\} \cup \{p \oplus b : b < q\},$$

hiểu theo các giá trị SG có thể đi tới từ từng trò con. Do đó mex của tập giá trị hậu duệ trong trò tổng chính là xor của hai giá trị SG.

Với n trò, áp dụng kết quả hai trò nhiều lần:

$$\begin{aligned} \text{SG}(x_1, \dots, x_n) &= \text{SG}(x_1, \dots, x_{n-1}) \oplus \text{SG}(x_n) \\ &= \text{SG}(x_1) \oplus \text{SG}(x_2) \oplus \dots \oplus \text{SG}(x_n). \end{aligned}$$

Tài liệu tham khảo

1. Theodore L. Turocy, Texas A&M University, *Game Theory*.
2. Erik D. Demaine, Robert A. Hearn, *Algorithmic Combinatorial Game Theory*.
3. Thomas S. Ferguson, *Game Theory*.
4. E. R. Berlekamp, J. H. Conway, R. K. Guy, *Winning Ways for Your Mathematical Plays*.
5. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*.